Reporte

Nombre: Valeria Ordaz Garcia Luis José Mendoza Serrano Omar Alejandro Salazar Castro	**Matrícula**: 2169894 2223173 2226576
Nombre del curso: Componentes de lógica del negocio	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Modulo Netacad:	**Actividad:** Diseño arquitectural - Elegir patrones
Fecha: 12/10/2026	
Bibliografia (Formato APA): Documentos impartidos por el docente Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley. Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.	

Objetivo:

Seleccionar y justificar el patrón de lógica de negocio que se utilizará en EduTechAcademy, comparando Transaction Script, Table Module y Domain Model con base en las reglas de negocio, restricciones y problemas identificados en el BRD y las User Stories.

Procedimiento:

Elección del patrón de lógica de negocio

Enfoque

Fowler (2002) describe tres formas principales de organizar la lógica de dominio de una aplicación empresarial: Transaction Script, Table Module y Domain Model. La elección depende sobre todo de qué tan compleja es la lógica y de cuánto cambia con el tiempo. Para decidir, primero se identificaron las características del BRD que condicionan la elección, después se definieron criterios de evaluación a partir de ellas y finalmente se compararon los tres patrones con una tabla.

Características del caso que condicionan la decisión

- **Muchas reglas que dependen unas de otras.** El BRD define 14 reglas de negocio (RB-01 a RB-14) y 7 restricciones (RES-01 a RES-07). La certificación, por ejemplo, depende de la calificación, la asistencia y el estado de la inscripción (RB-03, RB-06, RB-07 y RB-08).
- **Umbrales que se usan en varios procesos.** La calificación mínima de 70/100 y la asistencia mínima de 80 % intervienen en la consulta del estudiante (US-06), la validación de Control Escolar (US-13) y los reportes REP-06 y REP-07.
- **Invariantes que deben cumplirse en la misma operación.** Un grupo no puede rebasar su capacidad (RB-02), la inscripción no se confirma sin pago (RB-03) y un cambio de grupo exige cupo y autorización (RB-09).
- **Trazabilidad como requisito.** Los cambios de grupo y de instructor deben conservar su historial (RB-09, RB-11, RES-05 y CE-04).
- **Problemas actuales causados por lógica dispersa.** La validación manual de certificados (PA-01), los errores al consolidar asistencia y calificaciones (PA-02) y la dificultad para identificar candidatos (PA-04) vienen de que las reglas no están en un solo lugar.

- **Conceptos con relaciones entre sí.** Curso, Grupo, Inscripción, Estudiante e Instructor se relacionan de forma constante, y varios actores operan sobre ellos con permisos distintos (RB-14).
- **Una parte del sistema es solo consulta.** El catálogo de cursos y los reportes (REP-01 a REP-10) no modifican estados ni aplican reglas complejas.

Patrones considerados

Patrón	Idea principal (Fowler, 2002)	Funciona mejor cuando…	Debilidad principal
Transaction Script	Cada caso de uso se implementa como un procedimiento que ejecuta toda la lógica de principio a fin.	La lógica es simple y cada transacción es casi independiente de las demás.	Cuando la lógica crece, las reglas se duplican entre procedimientos y el código se vuelve difícil de mantener.
Table Module	Una sola instancia por tabla de la base de datos contiene la lógica de todas las filas de esa tabla.	La lógica es de complejidad media y el sistema ya trabaja con conjuntos de registros por tabla.	Las reglas que involucran varias tablas no tienen un lugar claro y no hay objetos individuales con identidad.
Domain Model	Un modelo de objetos representa los conceptos del negocio, con sus datos y su comportamiento.	La lógica es compleja, tiene muchas reglas interrelacionadas y cambia con frecuencia.	Mayor curva de aprendizaje, más código inicial y necesidad de mapear los objetos a la base de datos.

Nota: Active Record no se incluye en la comparación porque Fowler lo clasifica como un patrón de acceso a datos y no como una forma de organizar la lógica de dominio. Se menciona solo como una variante simplificada de Domain Model para casos de complejidad media.

Tabla comparativa

Cada criterio se calificó según qué tan bien lo cubre el patrón: **Alta**, **Media** o **Baja** adecuación para EduTechAcademy. La columna de criterio incluye la evidencia del BRD que lo motiva.

Criterio	Transaction Script	Table Module	Domain Model
1. Complejidad de las reglas (RB-01 a RB-14, RES-01 a RES-07)	**Baja.** Cada caso de uso se convierte en un procedimiento largo con muchas condiciones.	**Media.** Las reglas de una tabla se organizan bien, pero las que cruzan tablas quedan fuera.	**Alta.** Cada regla vive en el objeto que la posee (Grupo, Inscripción, Certificación).
2. Duplicación de umbrales (70/100 y 80 %; US-06, US-13, REP-06, REP-07; PA-02, PA-04)	**Baja.** El umbral se repite en cada script que valida o consulta.	**Media.** Se concentra por tabla, aunque no existe un concepto propio de "calificación".	**Alta.** Se define una sola vez en Calificación.aprueba() y PorcentajeAsistencia.cumpleMinimo().
3. Cambio de políticas (por ejemplo, subir la asistencia mínima)	**Baja.** Hay que localizar y modificar varios scripts.	**Media.** Se modifica una clase, pero la regla sigue atada a la estructura de la tabla.	**Alta.** Se modifica un solo Value Object.
4. Invariantes (cupo RB-02, pago RB-03, cambio de grupo RB-09)	**Baja.** Cada script debe volver a verificarlas manualmente.	**Media.** Se pueden validar por tabla, pero no entre Grupo e Inscripción.	**Alta.** Los Aggregates protegen sus invariantes en cada operación.
5. Historial y trazabilidad (RB-09, RB-11, RES-05, CE-04)	**Baja.** Requiere tablas de bitácora manejadas a mano.	**Baja.** Igual que Transaction Script.	**Alta.** CambioDeGrupo y AsignaciónInstructor son entidades del modelo.
6. Relaciones entre Curso, Grupo, Inscripción, Estudiante e Instructor	**Baja.** Las relaciones se resuelven con consultas dentro de cada script.	**Media.** Se manejan mediante conjuntos de registros relacionados.	**Alta.** Se expresan directamente como asociaciones entre objetos.
7. Lenguaje ubicuo (exigido en el mapa de dominio)	**Baja.** Los nombres reflejan procedimientos, no conceptos del negocio.	**Baja.** Los nombres reflejan tablas.	**Alta.** Clases y métodos usan los términos del glosario (Inscripción, Cupo, Certificación).
8. Pruebas unitarias de las reglas	**Baja.** Los scripts suelen depender de la base de datos.	**Media.** Se pueden probar por clase, pero con datos tabulares.	**Alta.** Las reglas se prueban con objetos en memoria, sin base de datos.

Criterio	Transaction Script	Table Module	Domain Model
9. Costo y curva de aprendizaje inicial	**Alta.** Es el más sencillo de implementar al inicio.	**Media.** Requiere cierta estructura de conjuntos de registros.	**Baja.** Exige más diseño y más código al inicio.
10. Encaje con lo pedido en la Unidad 2 (Aggregates, Value Objects, Domain Events, Service Layer)	**Baja.** No tiene un equivalente para estos conceptos.	**Baja.** Tampoco los incluye.	**Alta.** Es el patrón sobre el que se definen estos conceptos.
Puntuación total (Baja = 1, Media = 2, Alta = 3; sin ponderar)	**11 de 30**	**17 de 30**	**28 de 30**

La puntuación es solo un apoyo visual: la decisión se basa en los criterios 1 a 7, que corresponden a la naturaleza del problema. El criterio 9 es el único donde Domain Model queda por debajo, y se trata como un costo asumido en la sección de riesgos.

Decisión

Se utilizará Domain Model como patrón de lógica de negocio, con un Service Layer encima que coordina los casos de uso. Esta decisión se documentará como ADR-001.

Las razones principales son:

1. **La lógica del negocio es compleja y está interrelacionada.** Las reglas RB-01 a RB-13 dependen unas de otras, por lo que conviene que vivan en los objetos que las poseen y no en procedimientos independientes.
2. **Ataca la causa de los problemas actuales.** PA-02 y PA-04 aparecen porque la información y las reglas están dispersas. Concentrar el cálculo de asistencia, calificación y elegibilidad en el dominio da una sola fuente de verdad.
3. **Los umbrales quedan en un solo lugar.** Si la institución cambia el 70/100 o el 80 %, solo se modifica un Value Object.

4. **La trazabilidad se modela de forma natural.** Los cambios de grupo y las sustituciones de instructor son entidades con identidad e historial propio (RB-09, RB-11, CE-04).
5. **Es coherente con el resto de la Unidad 2.** Aggregates, Value Objects, Domain Events y Service Layer están pensados para funcionar sobre un Domain Model (Evans, 2003; Fowler, 2002).

Cómo se aplica en EduTechAcademy

El Domain Model no se usa para todo. Se aplica donde hay reglas de negocio y se evita donde solo se consulta información:

Parte del sistema	Patrón	Ejemplos
Reglas de negocio (cupo, requisitos, pago, criterios de certificación, historial)	Domain Model	Cupo.hayLugar(), Calificación.aprueba(), PorcentajeAsistencia.cumpleMinimo(), Inscripción.cambiarGrupo()
Casos de uso que modifican información	Service Layer sobre el Domain Model	InscripcionService.inscribir(), CertificacionService.validarCertificacion(). Solo coordinan; la decisión de negocio la toma el dominio.
Consultas de solo lectura	Consulta directa a repositorios, sin cargar Aggregates	CatalogoCursosService y ReporteService (REP-01 a REP-10).
Persistencia	Repositorios por Aggregate	CursoRepository, GrupoRepository, InscripcionRepository.

Por qué se descartaron los otros patrones

Transaction Script sería suficiente si el sistema solo registrara inscripciones y asistencia. Sin embargo, el BRD exige validar certificación con dos criterios, mantener historial y proteger invariantes entre varios conceptos. En ese escenario, los umbrales y las validaciones se repetirían en cada script, que es justo lo que hoy causa PA-02 y PA-04.

Table Module ordena mejor la lógica que Transaction Script, pero está pensado para trabajar con conjuntos de registros por tabla. EduTechAcademy necesita objetos con identidad propia (Inscripción, Grupo, CambioDeGrupo) y reglas que cruzan varios de ellos, algo que este patrón no maneja bien.

Costos y riesgos asumidos

Costo o riesgo	Impacto	Cómo se mitiga
Mayor curva de aprendizaje	El equipo debe dominar Aggregates, Value Objects y eventos.	Glosario de lenguaje ubicuo y documentación de decisiones en ADR.
Más código y diseño inicial	Se tarda más que con un Transaction Script.	Aplicar Domain Model solo a las partes con reglas, no a consultas ni reportes.
Diferencia entre objetos y base de datos	Los Aggregates no se corresponden uno a uno con las tablas.	Usar repositorios por Aggregate para ocultar el mapeo.
Posible sobreingeniería	Modelar de más conceptos de los necesarios.	Mantener Aggregates pequeños y basar cada elemento en una regla del BRD.